Autonomous Line-Following Vehicle with Integrated Obstacle and Color Detection System

Mohamed Abdo*, M Aamir Ejaz Ejaz*,Md Ruhul Kuddus Saleh*, Md Mostafizur Rahman*
*Systems Engineering and Prototyping, BSc Electronic Engineering
Hochschule Hamm-Lippstadt, Lippstadt, Germany
{mohamed-sayed-mohamed.abdo, muhammad-aamir.ejaz-ejaz, md-ruhul-kuddus.saleh, md-mostafizur.rahman}@stud.hshl.de

***Abstract*—In this study, our team's line-following vehicle prototype is shown. The robot can follow designated routes, recognize and detect obstacles, identify their color, and react appropriately based on that color. The project's objectives, hardware design, electronic components, and programming strategies are all examined in this article in order to accomplish the intended functionality. Additionally, it displays the tasks finished, tests conducted, and experiments conducted during the semester in order to evaluate and validate the efficacy of the system.**

***Index Terms*—Line follower robot,**

I. INTRODUCTION

This project presents an autonomous line-following vehicle capable of obstacle detection and color-based decision-making. Using an Arduino UNO R4, IR sensors, ultrasonic sensors, and a TCS3200 color sensor, the vehicle follows a black line and reacts to obstacles intelligently. It overtakes red objects, pushes green, and performs park maneuver if the object color is unknown. A servo-mounted ultrasonic sensor enhances obstacle scanning, while a voltage monitoring system adjusts speed for optimal performance. This project integrates embedded systems, sensor fusion, and robotics control, offering a practical model for intelligent navigation.

II. SYSTEM DESIGN AND ENGINEERING

The system design of the autonomous vehicle integrates hardware components and embedded logic to achieve efficient and adaptive navigation. The core controller, an Arduino UNO R4, manages sensor data processing and motor control. Two infrared sensors are positioned to detect the black line path, enabling continuous line-following behavior.

For obstacle detection, an ultrasonic sensor is mounted on a servo motor, allowing dynamic scanning of the surroundings. This rotating sensor increases detection accuracy and helps in planning overtaking maneuvers. A TCS3200 color sensor is used to identify the color of the obstacle, determining how the vehicle should react: when the car detects red, it starts to overtake, when it detects green, it pushes, and when it detects an unidentified object, it parks.

Motor control is achieved through an L298N motor driver, which provides bidirectional movement and turning. A voltage monitoring system adjusts the car's speed based on battery levels to maintain consistent performance. The coordination between sensors, actuators, and programmed logic forms a robust embedded control system for real-time decision-making.

III. PROTOTYPING AND DESIGN

A. Electronic Components

The vehicle's control system is centered around an Arduino UNO R4, which manages inputs from sensors and outputs to motors. Two infrared (IR) sensors are mounted at the front underside of the chassis to track the black line on the ground. An ultrasonic sensor, mounted on a vertical plate, provides distance measurements for obstacle detection. The TCS3200 color sensor is positioned close to the ground to accurately detect object color. An L298N motor driver is used for bidirectional control of the DC motors, and a voltage divider circuit is included to monitor battery levels in real-time.

B. Virtual and Physical Prototypes

The design process began with virtual prototyping using SolidWorks to visualize the mechanical structure and component placement. The virtual prototype allowed optimization of sensor angles, wheel spacing, and clearances before committing to physical assembly. After finalizing the CAD model, the physical prototype was built using a two-level chassis made from lightweight materials to separate the sensor and control layers. Multiple iterations were conducted to test wiring layout, sensor performance, and motor response.

C. Hardware Assembly

The hardware was mounted as per the CAD model. The ultrasonic sensor is fixed on a frontal panel with symmetrical alignment to maximize forward detection range. Below the base plate, the IR sensors are angled slightly downward for optimal ground visibility. The servo motor controlling the ultrasonic scanner is centrally positioned to sweep the surroundings effectively. All electronic components, including the Arduino board, motor driver, and battery pack, are mounted securely to minimize vibrations and maintain system stability during motion. The structured layout ensures efficient wiring, accessibility for maintenance, and protection of components during operation.

IV. SYSML DIAGRAMS

A. Requirement Diagram

The criteria for the autonomous vehicle system are shown in Figure 1 Line-following, dynamic speed control, and color-based obstacle handling are all supported. The Unknown

colors cause the car to park, green starts a push action, and red starts an overtaking maneuver. Figure 1 displays the requirements for the autonomous car system. Line-following, dynamic speed control, and color-based obstacle handling are all supported by the system. Green barriers are pushed aside, red starts a detour, and unidentified hues result in a parking maneuver. The sensor-driven, modular architecture of the design guarantees dependable and secure navigation.

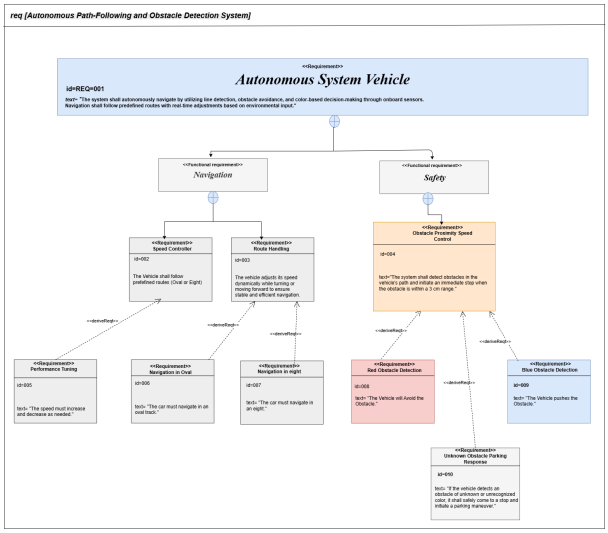

Fig. 1. Requirement Diagram

B. State Machine Diagram

A state machine model is used in Figure. 2 to depict the control logic of the autonomous vehicle in order to support this behavior. The system starts in idle mode and then switches to line-following navigation. The car branches its behavior based on color input when it detects obstacles: green starts a push action, red starts a detour, and unknown colors force the car to halt and park. When a task is finished or unrecoverable situations are encountered, the system enters a standstill state. In order to preserve operational safety, emergency transitions also manage situations like line loss.

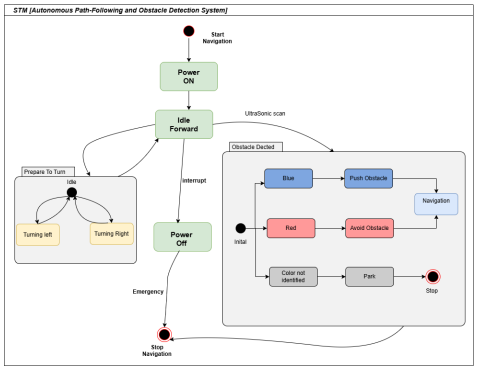

Fig. 2. State Machine Diagram

C. Activity Diagram

The activity diagram in Figure. 3shows the autonomous vehicle's operating flow in accordance with the state machine logic. By modifying its motion in response to line detection, the system initiates autonomous navigation. It continuously recognizes and assesses the color of obstacles it detects as it moves forward. Unknown colors cause the car to stop and park, green starts a push maneuver, and red starts a detour that is followed by a return to the main path. Until the route is successfully finished or the system is manually stopped, this process keeps on.

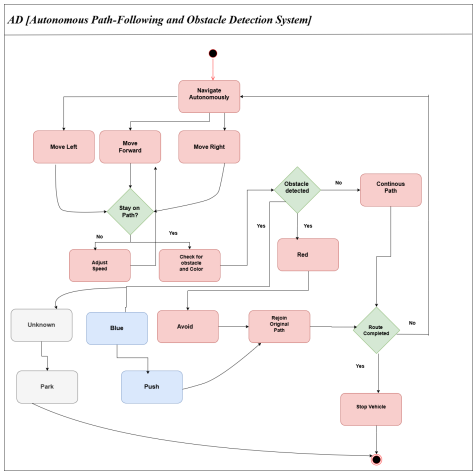

Fig. 3. Activity Diagram

D. Use Case Diagram

The use case diagram Figure. 4 depicts the main characteristics of the Autonomous Path-Following and Obstacle Detection System. As the primary external actors, the team members engage with the system by turning it on and off using the switch on/off feature. The technology automatically navigates once it is turned on, detecting lines, obstacles, and regulating speed. Using line-following logic, the navigation process facilitates directional decisions like turning left or right. The system uses obstacle detection to make color-based decisions: unknown colors, as specified by the modified system behavior, force the vehicle to stop and park; green obstacles commence a push maneuver; and red obstacles start an avoidance or detour sequence. Furthermore, under the speed-regulated navigation mode, the system can follow predefined path patterns like oval and figure-eight routes.

E. Sequence Diagram

The sequence diagram Figure.5 shows how the various parts of the system interact during autonomous navigation. Line detection and sensor activation are started when the user turns on the system. The car continuously looks for obstructions as it travels forward. Green obstacles are pushed aside, red obstacles cause an overtaking action, and unknown colors cause the system to halt and park. The system employs the color sensor to classify obstacles when it detects them. In

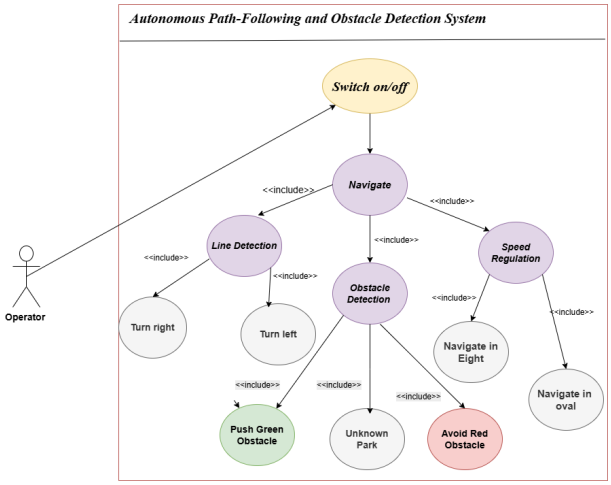

Fig. 4. Use Case Diagram

the meantime, motor control uses input from the servo's area scan to dynamically modify speed and direction. Smooth, autonomous navigation is made possible by this time-sequenced interaction, which guarantees real-time decision-making.

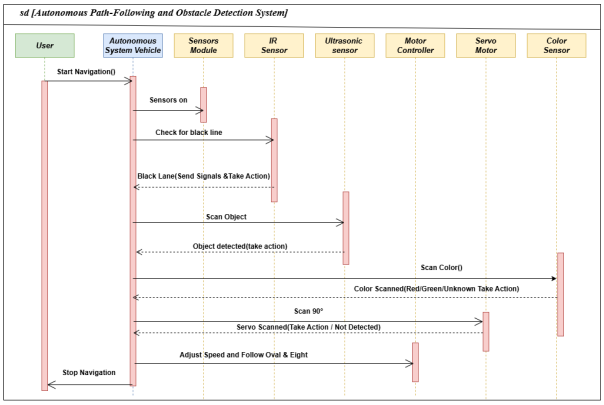

Fig. 5. Sequence Diagram

V. Structure Diagrams

The autonomous vehicle system's static architecture is specified via structure diagrams. They show how hardware and software components are arranged, relate to one another, and work together to support navigation, obstacle detection, and decision-making features.

A. Block Definition Diagram

Moving onto the structure part of the SysML,we start with the Block Definition Diagram (BDD). The BDD in Figure. 6 is used to model the structural composition of the autonomous vehicle system, emphasizing how the system is organized and broken down into functional parts. It demonstrates how the two main subsystems safety and navigation are further broken down into essential elements like line detection, obstacle avoidance, and route selection. Because these functional blocks are linked to hardware elements like motors, sensors, and the Arduino UNO, they show how software logic is translated onto the physical architecture. The way that different components work together to enable the vehicle's autonomous behavior is clearly understood thanks to this structural representation.

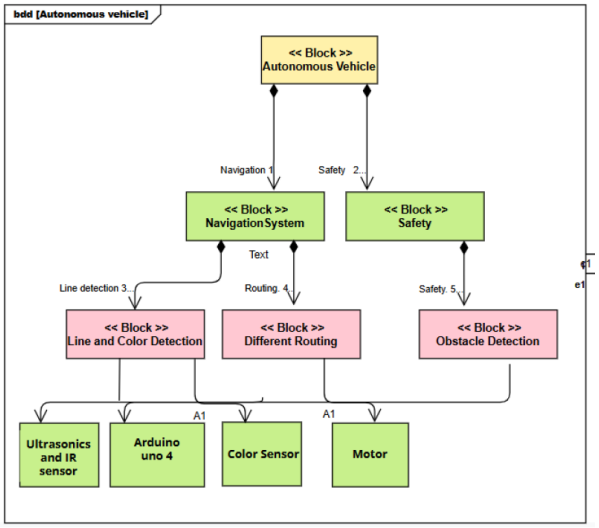

Fig. 6. Block Definition Diagram

B. Package Diagram

The autonomous vehicle system's modular design is depicted in the package diagram Figure. 7 . It divides related tasks into logical subsystems, including obstacle handling, color detection, and navigation. Sensors, actuators, and system logic are all coordinated by the Arduino UNO, which serves as the main controller. All active components are supported by the power supply, guaranteeing continuous functioning. The control, sensor, and actuation layers are neatly divided in this system.

C. Internal Body Diagram

The autonomous vehicle's internal organization and communication flow are depicted in the internal block diagram Figure. 8 . The Line and Color Detection device processes sensor data from infrared, ultrasonic, and color sensors and sends pertinent signals to the Arduino UNO. Coordinated tire movement is the consequence of the Arduino controlling the motor depending on the processed inputs. This configuration demonstrates how perception, control, and actuation elements can be integrated.

D. Parametric Diagram

The autonomous vehicle system's dependencies and limitations are depicted in the parametric diagram Figure. 9 It demonstrates how important features like line following and obstacle identification are influenced by sensor inputs and preset factors like track type and obstacle behavior. As the primary controller, the Arduino UNO processes inputs from infrared and ultrasonic sensors and regulates tire movement in response to system circumstances.

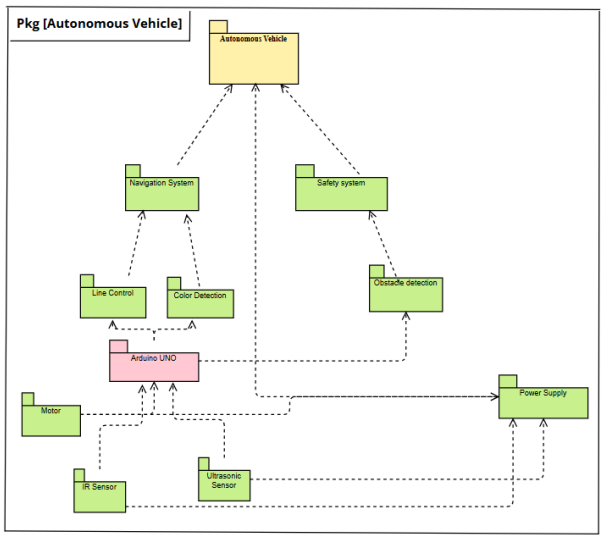

Fig. 7. Package Diagram

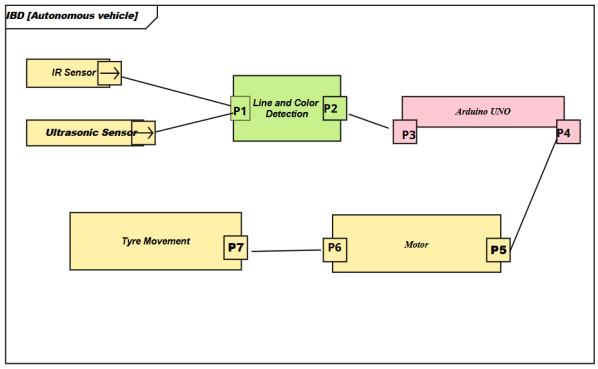

Fig. 8. Internal Body Diagram

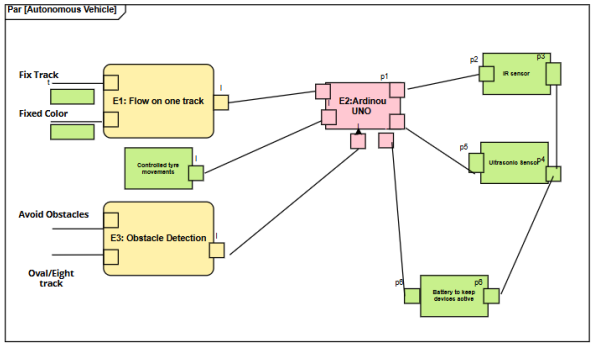

Fig. 9. Parametric Diagram

VI. Autonomous Navigation Simulation

We ran a number of simulations, starting with the electronic circuitry, to confirm the suggested autonomous car system's operation prior to its actual deployment. We created and modelled the main electrical circuit using Multisim in order to confirm sensor integration, motor driver behavior, and power distribution.

A. Multisim Simulation

The autonomous vehicle's circuit diagram is displayed in Figure. 10. An LM7805CT voltage regulator is used to regulate the system's 11.1 V battery to 5 V so that all of its parts can operate safely. The wheels are driven by two DC motors that are managed by a motor driver. Two infrared sensor pairs each comprising an IR LED and phototransistor as well as pull-up and current limiting resistors for signal stability are used to accomplish line-following. While a color sensor recognizes the colors of objects, an ultrasonic sensor finds impediments. The ultrasonic sensor is rotated for scanning by a servo motor. The circuit has a switch to turn on the system and capacitors to smooth out power.With this configuration, the car can assess its surroundings and decide how to travel in real time.

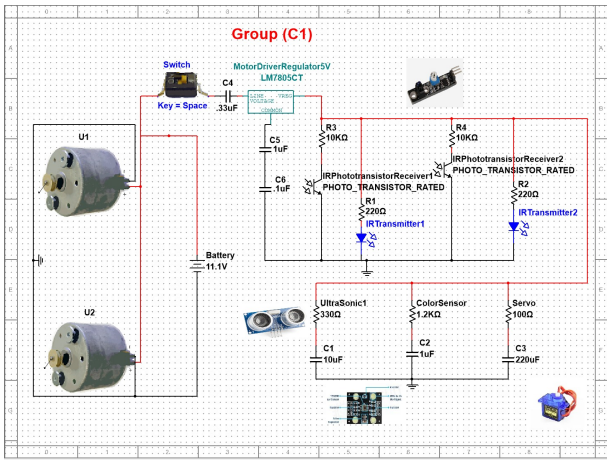

Fig. 10. Multisim simulation

B. Tinkercad Simulation

Tinkercad, a virtual electronics platform that enables circuit design and testing without actual components, was used to simulate the hardware implementation of the autonomous car system Figure. 11 An H-bridge motor driver is used in the simulation to connect an Arduino UNO microcontroller to two DC motors, enabling directional control. IR sensors are placed for white line tracking, and an ultrasonic sensor (HC-SR04) is employed for obstacle detection. A 9V battery powers the system, and a breadboard is used in the setup to control connections and power distribution. Before the system is physically deployed, the simulation verifies its viability in the real world by validating motor control logic, sensor responses, and navigation behavior.

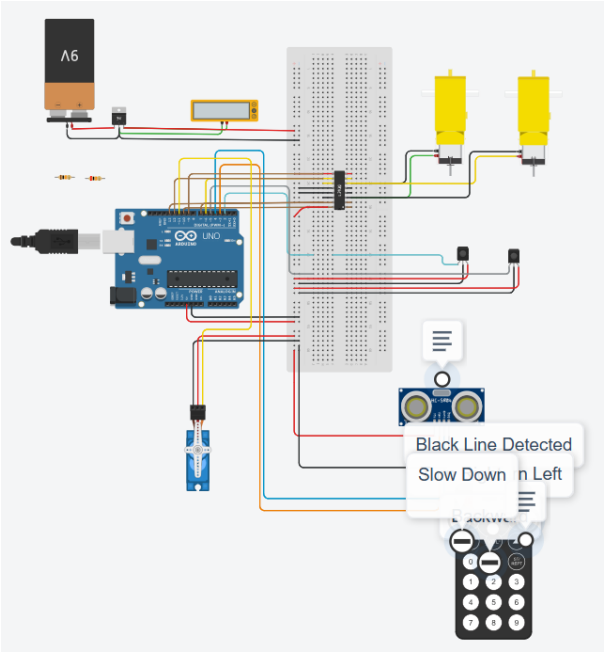

Fig. 11. Tinkercad simulation

C. Schematic View

A detailed schematic diagram Figure. 12 was created to depict the real hardware wiring of the autonomous car system after the Tinkercad simulation. All necessary electrical connections and component interconnections are depicted in the schematic. The central component is the Arduino UNO microcontroller (U1), which communicates with a servo motor (SERVO1) for scanning, an ultrasonic sensor (DIST1) for obstacle detection, and an H-Bridge motor driver (U5) for bidirectional motor control. A voltage regulator (U4) controls a 9V battery to provide steady 5V output. The direction movement is made possible by the motor driver, which controls the two DC motors (M1 and M2). For improved sensing and signal dependability, further modules are integrated, such as resistors and color sensors (U2, U3). Before the circuit is physically assembled, this schematic guarantees a precise implementation.

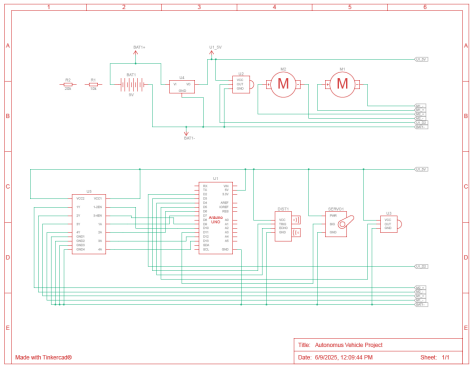

Fig. 12. Schematic View

VII. UPPAAL-Based Behavioral Modeling and Simulation

We used UPPAAL, a real-time system modeling tool for validating and verifying timed automata, to model the vehicle's decision-making logic in order to guarantee dependable and secure behavior in autonomous navigation. The model is made up of multiple interacting timed automata, each of which is in charge of a distinct functional component of the behavior of the autonomous car.

A. Main Navigation Controller

The primary controller automaton in charge of guiding the vehicle's fundamental movement behaviors is depicted in the Figure. 13. After the Move trigger, it moves from an initial SwitchOn state to MoveForward. The automata switches between states like TurnLeft, TurnRight, Stop, and obstacle-related handling states like ColourCheck, OvertakeObstacle, and MoveTheObstacle in response to different sensor inputs. To ensure context-aware reactions, events like RedObstacle, GreenObstacle, or UnknownObstacle! initiate each transition.

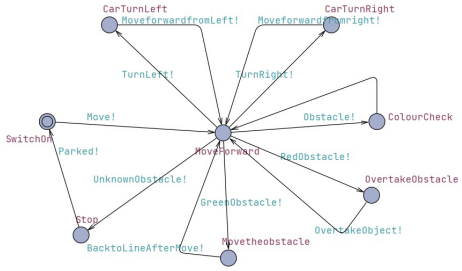

Fig. 13. Main Navigation Controller

B. Obstacle Detection and Response Logic

When the system identifies a potential obstacle, it switches from the Obstacle free state to the Measuring Distance state (Move?) Figure. 14. It transitions to the Obstacle Detected state and then to Colour Check to determine the object's color if an obstacle is verified (Obstacle?). The system determines what to do next, If the obstacle is red, the vehicle initiates an overtaking maneuver. If the obstacle is green, it performs a pushing action. For unknown color obstacles, the vehicle decides to park aside. The system reverts to the Obstacle free state and resumes regular function once the impediment has been effectively avoided or eliminated.

C. Left Side Behavior

The behavior of the left-side IR sensor is modeled by this automata. When the vehicle is not on line (CarNotOnLine), it turns on, checks the LeftIRon condition, makes a left correction (TurnLeft?) and then switches back to MoveForward Figure. 15.

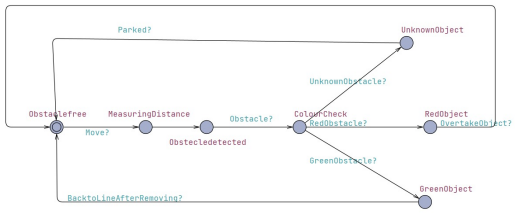

Fig. 14. Obstacle Detection and Response Logic

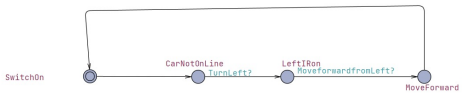

Fig. 15. Left Side Behavior

D. Right Side Behavior

The opposite of the left IR behavior is this Figure. 16. Before going back to MoveForward, it reacts to CarNotOnLine, checks the RightIRon signal, and orders a right turn (TurnRight?).

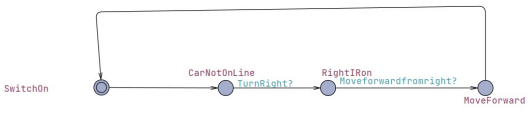

Fig. 16. Right Side Behavior

VIII. 3-D CAD Model of the Autonomous Car Prototype

The completed 3D CAD design of the autonomous vehicle created for this project is shown in Figure. 17 Throughout the building process, the CAD model was a vital source of information that guaranteed precise component placement and spatial alignment. Important hardware elements like dual-wheel differential drive systems, onboard controller mounts, ultrasonic sensors, and DC motors with encoders are all integrated into the design. Notably, the front panel sensor mount facilitates ambient awareness and obstacle recognition, both of which are critical for autonomous navigation activities. This design is carefully followed in the physical assembly, demonstrating the value of CAD modeling in system integration and iterative prototyping.

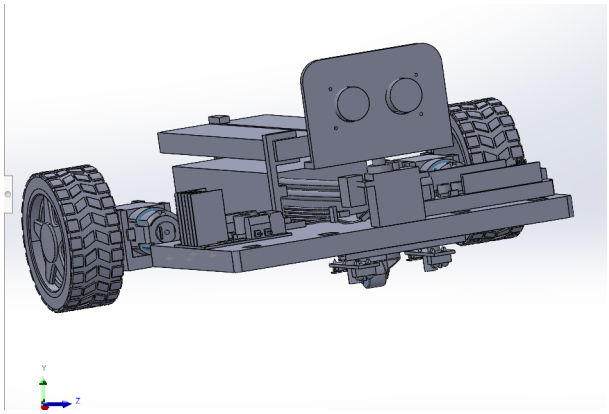

Fig. 17. 3-D CAD Model

IX. Electronics Parts Used

An Arduino Uno microcontroller serves as the main component of the autonomous car's electronics system, interacting with a variety of sensors and actuators. A TCS3200 color sensor for object categorization, an ultrasonic sensor for obstacle detection, and infrared sensors for line following are important parts. For directed scanning, the ultrasonic sensor is rotated by a servo motor. A 7.4 V or 11.1 V LiPo battery powers the L298N twin H-bridge motor driver, which controls the motors. Sensitive components receive a steady 5 V power supply thanks to a voltage regulator. A breadboard and jumper wires link all the components, allowing for modular and test-friendly assembly.

A. Arduino Uno R4

The Arduino Uno R4 Figure. 18 serves as its main controller, gathering input from sensors, processing it, and using actuators to initiate the required steps to allow movement. It is based on the Atmega328 microcontroller, which has an 8-bit RISC processor core and is a member of the Atmel family. As seen in Figure 16, the board has six analog input pins and fourteen digital GPIO pins. It communicates with a PC over a USB interface.The Arduino Uno is notable for its Atmega328P microcontroller, which has six analog input pins, 14 digital GPIO ports, and an 8-bit RISC CPU core. Because of these features, the Arduino Uno can effectively manage sensor inputs and control actuators, which makes it an essential part of the prototype's operation. [1]

B. Ultrasonic Sensor

The Ultrasonic sensors Figure. 19 with a resolution of about 3 mm via ultrasonic signaling, this module is ideal for measuring distances ranging from 2 cm to 3 meters. The sensor starts measuring distance when a falling edge signal is applied to the trigger input. The distance measurement is then output as a PWM TTL signal from the echo pin. Applications including obstacle detection, distance measuring, liquid level monitoring, and other industrial uses are where the ultrasonic distance sensor excels. When activated, the ultrasonic transducer produces a 200 s burst of ultrasonic

Fig. 18. Arduino Uno R4

waves at a frequency of 40 kHz, which produces 8 signal cycles at that time. A 10 s high-level pulse applied to the trigger input pin causes this emission. The echo output pin then swings high after this. The echo pin is pulled low when the microphone receives the reflected ultrasonic wave.The ultrasonic wave's round-trip travel time from the transducer to the reflecting item and back is represented by the amount of time the echo pin stays high. It is possible to compute the distance to the object (as half the total traveled distance) by measuring this duration and assuming that sound travels at a constant speed in air. [2]

Fig. 19. Ultrasonic Sensor

C. IR Sensor

We make use of photodiodes, which are infrared (IR) transmitters and receivers Figure. 20 The photodiode serves as the receiver, identifying the reflected infrared signals and transmitting the relevant data to the microcontroller. The IR transmitter is an LED that emits infrared light. A detectable voltage shift occurs when the infrared light strikes a white surface because it is reflected back and picked up by the photodiode. On the other hand, when infrared light hits a black surface, it is absorbed instead of reflected, and the photodiode doesn't get any signal [3].

Fig. 20. IR Sensor

D. Color Sensor

An array of photodiodes filtered for red, green, blue, and clear (unfiltered) light is how the TCS3200 color sensor works Figure. 21. The 4x4 matrix arrangement of these photodiodes enables precise measurement of light intensity throughout the visible spectrum. A current-to-frequency converter built inside the sensor produces a square wave signal whose frequency is proportional to the color's intensity. The sensor can separate and measure the intensity of red, green, or blue light separately since the selection of particular color channels is digitally controlled. Depending on the color composition of the light, the matching photodiodes produce current when it reflects off a surface and enters the sensor. After that, this current is transformed into a frequency signal that a microcontroller can read and process to ascertain the surface's hue. Because of this feature, the TCS3200 can be used in a number of applications, such as embedded robotics, industrial sorting systems, and color identification [4].

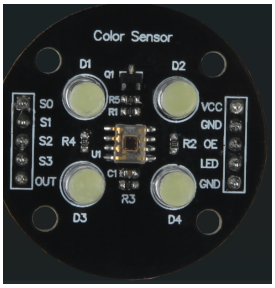

Fig. 21. Color Sensor

E. Motor Controller

The L298N is a dual-channel H-bridge motor driver that can provide both speed and direction control for two DC motors or one stepper motor Figure. **??**. It can withstand peak currents of up to 2 A per channel and take motor supply voltages ranging from 5 V to 35 V (with an absolute maximum of 46 V).Two enable pins (ENA and ENB) receive PWM signals to control speed, and two logic inputs (IN1/IN2 for Motor A and IN3/IN4 for Motor B) control each H-bridge section by determining motor direction. If the input voltage is 12 V or less, the module's onboard 78M05 linear regulator, which supplies a 5 V logic supply, can also power an external microcontroller.Integrated flyback diodes to guard

against inductive voltage spikes and thermal shutdown to stop overheating are examples of protection features. For robotics, industrial actuation, and applications that require bidirectional motor control with changing speed, this makes the L298N module ideal [5].

Fig. 22. Motor Controller

F. Lippo Battery

A Conrad 7.4V 3000mAh 20C Eco-Line LiPo battery, illustrated in Figure. 23 , was used to supply power to the L298N motor driver. According to the manufacturer, the battery weighs approximately 210 grams [6].

Fig. 23. Lippo Battery

X. Software Implementation

A. Initialization of Sensors and Actuators

To get the system ready for use, all the sensors and actuators are initialized in the Setup function of the Arduino code. This entails specifying the pin modes for servo motors, motors, color sensors, ultrasonic sensors, and infrared (IR) sensors. Depending on its purpose, each component is set up as either an input or an output. Additionally, serial communication is set up for monitoring and debugging. Pins for sensors were defined and initialized in the 'setup()' function. Below is a snippet:

Listing 1. Sensor Initialization Code

```
void setup() {
  Serial.begin(9600);
// Initialize serial communication

  // IR sensors
  pinMode(R_S, INPUT);
// Right IR sensor
  pinMode(L_S, INPUT);
// Left IR sensor

  // Motor control pins
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);
  pinMode(enB, OUTPUT);

  // Ultrasonic sensor
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  // Color sensor
  pinMode(S0, OUTPUT);
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  pinMode(S3, OUTPUT);
  pinMode(colorOut, INPUT);
  digitalWrite(S0, HIGH);
  digitalWrite(S1, LOW);

  // Servo motor
  scanServo.attach(servoPin);
  scanServo.write(50);
// Start at center position
  delay(300);
}
```

B. Movement Control Algorithms

Both programmed decision-making logic and sensor feedback regulate the autonomous vehicle's movement. A black line on the track is followed by infrared (IR) sensors. In order to maintain alignment, the algorithm continuously examines the values from the left and right infrared sensors. If both detect the line, the robot advances; if only one does, it turns in that direction. An ultrasonic sensor tracks the separation between barriers.The car pauses and activates color detection when it detects an object within a predetermined threshold. Different operations are carried out based on the detected color: blue starts a parking routine by reversing, turning, and halting; green commences a forward-and-reverse bypass; and red initiates a right-side diversion and rejoining of the line. PWM signals are used to dynamically modify motor speed, ensuring smooth turning and obstacle-handling. The robot can move in both reactive and adaptable ways on its own thanks to these control systems.

C. Behavioral Decision Logic

The way the car changes states in response to sensor input is controlled by the behavioral decision logic. Under typical circumstances, the car uses infrared sensors to follow the line. The car shifts into "approach" mode, cautiously approaching an obstruction when the ultrasonic sensor detects it within a predetermined distance. The system stops when it reaches a closer threshold and classifies the object using the color sensor.Depending on the color it senses, the robot does a certain action: blue parks the car and stops its motion, green is ignored after a brief sidestep, and red initiates a diversion.

If the line is lost at any of these stages, the system resets and starts an emergency pause. This decision logic ensures that the robot can adapt its actions dynamically to changes in the surroundings.

XI. Final Autonomous Car Prototype

The autonomous car's final prototype, depicted in Figure. 24 combines a number of sensors and electronic parts on a robotic platform with two wheels. The system is constructed on a wooden chassis and contains two DC motors that power the differential drive. A geared motor mount connects each motor to the wheels. By monitoring the surroundings, an ultrasonic sensor mounted on a servo motor at the front allows for dynamic obstacle detection. A TCS3200 color sensor is mounted directly beneath it to identify and categorize the colors of objects along the route. Infrared sensors positioned beneath the chassis and oriented to track black-and-white path transitions are used to implement line-following capabilities. Component integration and wiring are made easier by a single breadboard, which also makes connection testing and modification simple. An Arduino Uno manages the control logic, processing sensor inputs and adjusting actuators as necessary. The IR modules, ultrasonic sensor, color sensor, and motor driver (L298N), which is powered by a 7.4V 3000 mAh LiPo battery installed on the chassis, are among the several sensors that are connected via the wire. The arrangement is nonetheless modular and test-friendly, providing flexibility for upcoming enhancements or debugging even though the wiring is complicated due to the large number of interconnected components. Real-time experimentation and formal verification using UPPAAL models were used to validate this entire hardware system.

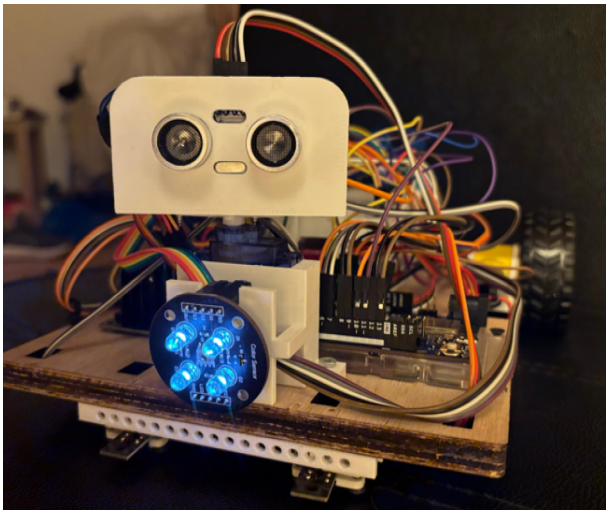

Fig. 24. Final Prototype

XII. Git Usage and Collaboration

Project development was tracked using GitHub. Each member's contribution should be explained.

Team Member	Number of Commits
Mohamed Abdo	34
Md Ruhul Kuddus Saleh	24
M Aamir Ejaz Ejaz	11
Md Mostafizur Rahman	10
Total	**114**

TABLE I
GitHub Commits by Team Members

XIII. Conclusion and Future Work

We successfully created an autonomous vehicle in this project that can identify the color of objects, follow lanes, and make decisions based on context. In addition to responding appropriately to recognized barriers based on their color starting an overtaking movement for red obstacles, pushing green objects, and parking aside for unknown colors the automobile consistently recognizes lane borders. These characteristics show how sensor inputs, real-time control logic, and fundamental autonomy can be integrated functionally. By switching from simple microcontroller platforms to more sophisticated hardware, such the Raspberry Pi, we hope to improve the system in subsequent work. This will allow for more processing power, adaptability, and the incorporation of machine learning models. In order to bring the system closer to real-world deployment standards, we also intend to add more professional-grade functionality and enhance the software architecture for improved scalability.

XIV. Limitations and Sensor Calibration Challenges

A significant problem with the color sensor's capacity to identify the green spectrum was discovered during testing. In particular, the sensor was unable to identify green because of an inadequate output frequency, which needs to be kept at 100 percent (S0=H, S1=H) for precise detection see Fig 25. The issue most likely stems from power constraints that impact the output frequency scaling of the sensor. Specifically for the green photodiode option (S2=H, S3=H), the system needs an external power supply to guarantee a steady and dependable output frequency for accurate color discrimination. This emphasizes how crucial proper power management and calibration are for applications that rely on sensors.

S0	S1	OUTPUT FREQUENCY SCALING (f_o)
L	L	Power down
L	H	2%
H	L	20%
H	H	100%

S2	S3	PHOTODIODE TYPE
L	L	Red
L	H	Blue
H	L	Clear (no filter)
H	H	Green

Fig. 25. Color Table

Acknowledgment

This project was made possible through the invaluable support, resources, and expertise provided by Hamm-Lippstadt University of Applied Sciences. We would like to extend our sincere gratitude to Prof. Dr. Stefan Henkler, Prof. Faezeh

Pasandideh, and Mr. G. Wahrmann for their unwavering guidance, commitment, and constructive feedback, which played a crucial role in the continuous improvement of our work.

REFERENCES

[1] "Uno R3," Arduino. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3/. [Accessed: Jun. 27, 2024].
[2] Joy-IT, "KY-050 ultrasonic distance sensor," SensorKit. [Online]. Available: https://sensorkit.joy-it.net/en/sensors/ky-050. [Accessed: Jun. 27, 2024]
[3] https://www.irjet.net/archives/V7/i2/IRJET-V7I2379.pdf
[4] "TCS3200 Programmable Color Light-to-Frequency Converter," Texas Advanced Optoelectronic Solutions (TAOS), [Online]. Available: https://www.mouser.com/catalog/specsheets/tcs3200-e11.pdf. [Accessed: Jun. 27, 2024]
[5] L298N Dual H-Bridge Motor Driver, Handson Technology. [Online]. Available: https://www.handsontec.com/dataspecs/L298N
[6] "Buy Conrad Energy Scale Model Battery Pack (LIPO) 7.4 V 5500 mAh no. of cells: 2 20 C Softcase XT90," Conrad Electronic. [Online]. Available: https://www.conrad.com/en/p/conrad-energy-scale-model-battery-pack-lipo-7-4-v-5500-mah-no-of-cells-2-20-c-softcase-xt90-1344152.html. [Accessed: Jun. 27, 2024]

DECLARATION OF ORIGINALITY

We hereby declare that this report is our own work and that we have acknowledged all sources used.